Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК5 «Системы обработки информации»***

Лабораторная работа № 4

«Метрики качества для задач классификации»

ДИСЦИПЛИНА: «Технологии обработки больших данных»

Выполнил: студент гр.ИУК5-82Б _____ (Ли Р.В. _____)
(Подпись) (Ф.И.О.)

Проверил: _____ (_____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

Калуга, 2026

Цель лабораторной работы — изучить ансамблевые методы бэггинга и градиентного бустинга, а также на практике сравнить реализации бустинга (XGBoost, LightGBM, CatBoost) с RandomForest на задачах классификации и регрессии, включая базовую предобработку и подбор гиперпараметров.

Задачи:

- понять идею бэггинга (бутстреп-выборки, независимое обучение базовых моделей, агрегирование ответов) и почему он снижает дисперсию;
- разобрать идею градиентного бустинга (последовательное добавление моделей, минимизация функции потерь по антиградиенту);
- изучить ключевые отличия XGBoost / LightGBM / CatBoost (рост деревьев, работа с пропусками, особенности категориальных признаков, важность признаков);
- выполнить практику: обучить модели на задаче классификации, сравнить по F1 и PR AUC, затем выполнить пример регрессии и сравнить модели по MSE/MAE/R^2;

выполнить GridSearchCV для подбора гиперпараметров (минимум для 2–3 моделей) и сравнить качество/скорость.

Классификация:

1. Загрузка и форматирование данных

```
[1]: import pandas as pd

     df = pd.read_csv("mushrooms.csv")
```

```
[4]: df = df.replace('?', None)
     from sklearn.preprocessing import LabelEncoder

     le = LabelEncoder()

     for col in df.columns:
         df[col] = le.fit_transform(df[col].astype(str))
```

```
[5]: from sklearn.model_selection import train_test_split

     X = df.drop('class', axis=1)
     y = df['class']
     X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, random_state=42
     )
```

2. Создание и обучение модели классификатора

```
[6]: from sklearn.ensemble import RandomForestClassifier

     rf = RandomForestClassifier()
     rf.fit(X_train, y_train)
```

[6]:

RandomForestClassifier

▾ Parameters

n_estimators	100
criterion	'gini'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	None
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

3. Создание и обучение модели XGBClassifier

```
[8]: from xgboost import XGBClassifier

     xgb = XGBClassifier(eval_metric='logloss')
     xgb.fit(X_train, y_train)
```

```
[8]: ▸ XGBClassifier
```

4. Создание и обучение модели LGBMClassifier

```
[8]: from lightgbm import LGBMClassifier

     lgbm = LGBMClassifier()
     lgbm.fit(X_train, y_train)
```

```
[LightGBM] [Info] Number of positive: 3134, number of negative: 3365
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000194 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 116
[LightGBM] [Info] Number of data points in the train set: 6499, number of used features: 21
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.482228 -> initscore=-0.071118
[LightGBM] [Info] Start training from score -0.071118
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[8]: ▾ LGBMClassifier
     ▸ Parameters
```

5. Создание и обучение модели CatBoostClassifier

```
[9]: from catboost import CatBoostClassifier

     cat = CatBoostClassifier(verbose=0)
     cat.fit(X_train, y_train)
```

```
[9]: CatBoostClassifier(verbose=0)
```

6. Сбор результатов обучения

[10]:
```python
from sklearn.metrics import f1_score, average_precision_score

models = {
    "RandomForest": rf,
    "XGBoost": xgb,
    "LightGBM": lgbm,
    "CatBoost": cat
}

for name, model in models.items():
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    f1 = f1_score(y_test, y_pred)
    pr_auc = average_precision_score(y_test, y_proba)

    print(f"{name}: F1={f1:.4f}, PR AUC={pr_auc:.4f}")
```

```
RandomForest: F1=1.0000, PR AUC=1.0000
XGBoost: F1=1.0000, PR AUC=1.0000
LightGBM: F1=1.0000, PR AUC=1.0000
CatBoost: F1=1.0000, PR AUC=1.0000
```

7. Подбор конфигурации для RandomTreeClassifier

[12]:
```python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier()

rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5]
}

rf_grid = GridSearchCV(
    rf,
    rf_params,
    cv=3,
    scoring='f1',
    n_jobs=-1
)

rf_grid.fit(X_train, y_train)

print("RF best params:", rf_grid.best_params_)
print("RF best score:", rf_grid.best_score_)
```

```
RF best params: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 100}
RF best score: 1.0
```

8. Подбор конфигурации для XGBClassifier

[13]:
```
from xgboost import XGBClassifier

xgb = XGBClassifier(eval_metric='logloss')

xgb_params = {
    'n_estimators': [100, 200],
    'max_depth': [3, 6],
    'learning_rate': [0.01, 0.1]
}

xgb_grid = GridSearchCV(
    xgb,
    xgb_params,
    cv=3,
    scoring='f1',
    n_jobs=-1
)

xgb_grid.fit(X_train, y_train)

print("XGB best params:", xgb_grid.best_params_)
print("XGB best score:", xgb_grid.best_score_)
```

```
XGB best params: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100}
XGB best score: 1.0
```

9. Подбор конфигурации для CatBoostClassifier

[14]:
```
from catboost import CatBoostClassifier

cat = CatBoostClassifier(verbose=0)

cat_params = {
    'depth': [4, 6],
    'learning_rate': [0.01, 0.1],
    'iterations': [200, 500]
}

cat_grid = GridSearchCV(
    cat,
    cat_params,
    cv=3,
    scoring='f1',
    n_jobs=-1
)

cat_grid.fit(X_train, y_train)

print("CatBoost best params:", cat_grid.best_params_)
print("CatBoost best score:", cat_grid.best_score_)
```

```
CatBoost best params: {'depth': 4, 'iterations': 200, 'learning_rate': 0.1}
CatBoost best score: 1.0
```

10. Вывод результатов оптимизации

[15]:
```
best_models = {
    "RF": rf_grid.best_estimator_,
    "XGB": xgb_grid.best_estimator_,
    "CatBoost": cat_grid.best_estimator_
}

from sklearn.metrics import f1_score, average_precision_score

for name, model in best_models.items():
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    print(name)
    print("F1:", f1_score(y_test, y_pred))
    print("PR AUC:", average_precision_score(y_test, y_proba))
    print()
```

```
RF
F1: 1.0
PR AUC: 1.0

XGB
F1: 1.0
PR AUC: 1.0

CatBoost
F1: 1.0
PR AUC: 1.0
```

Регрессия:

1. Загрузка данных и создание тренировочной и тестовой датасетов

```
[1]: import pandas as pd

     df = pd.read_csv("train.csv")
     print(df.shape)

     (30471, 292)

[2]: y = df["price_doc"]
     X = df.drop(["price_doc", "id", "timestamp"], axis=1)

[3]: num_cols = X.select_dtypes(include=['int64', 'float64']).columns
     cat_cols = X.select_dtypes(include=['object', 'string']).columns

[4]: # числовые
     X[num_cols] = X[num_cols].fillna(X[num_cols].median())

     # категориальные
     X[cat_cols] = X[cat_cols].fillna("missing")

[5]: X = pd.get_dummies(X)

[6]: from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, random_state=42
     )
```

2. Создание и обучение RandomForestRegressor

```
[9]: from sklearn.ensemble import RandomForestRegressor

     rf = RandomForestRegressor(
         n_estimators=100,
         max_depth = 10,
         n_jobs = -1
     )
     rf.fit(X_train, y_train)

[9]: ▾ RandomForestRegressor
     ▸ Parameters
```

3. Создание и обучение XGBRegressor

```
[10]: from xgboost import XGBRegressor

      xgb = XGBRegressor()
      xgb.fit(X_train, y_train)

[10]: ▾ XGBRegressor
      ▸ Parameters
```

4. Создание и обучение LGBMRegressor

```
[11]: from lightgbm import LGBMRegressor

      lgbm = LGBMRegressor()
      lgbm.fit(X_train, y_train)
```

```
[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.021990 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 41086
[LightGBM] [Info] Number of data points in the train set: 24376, number of used features: 440
[LightGBM] [Info] Start training from score 7146571.364170
```

```
[11]: ▾ LGBMRegressor ⓘ
      ▸ Parameters
```

5. Создание и обучение CatRegressor

```
[14]: from catboost import CatBoostRegressor

      cat = CatBoostRegressor(verbose=0)
      cat.fit(X_train, y_train)
```

```
[14]: CatBoostRegressor(loss_function='RMSE', verbose=0)
```

6. Вывод результатов обучения

```
[15]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

models = {
    "RF": rf,
    "XGB": xgb,
    "LGBM": lgbm,
    "CatBoost": cat
}

for name, model in models.items():
    preds = model.predict(X_test)

    mse = mean_squared_error(y_test, preds)
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)

    print(name)
    print("MSE:", mse)
    print("MAE:", mae)
    print("R2:", r2)
    print()
```

```
RF
MSE: 6751881176979.154
MAE: 1417305.0113338542
R2: 0.6845061391151137

XGB
MSE: 6747448672256.0
MAE: 1389165.0
R2: 0.6847132444381714

LGBM
MSE: 6368923242859.388
MAE: 1362340.0488761545
R2: 0.7024005412861545

CatBoost
MSE: 5998851304293.287
MAE: 1323496.3384236628
R2: 0.7196928220065937
```

Вывод: были изучены ансамблевые методы бэггинга и градиентного бустинга, а также на практике сравнить реализации бустинга (XGBoost, LightGBM, CatBoost) с RandomForest на задачах классификации и регрессии, включая базовую предобработку и подбор гиперпараметров.